

CSE 341 Final Project Proposal

Backend API Architecture Specification Blueprint

General Info

Student Names in group:

Alexander Bredee Holden, Eric Ayanru, Favour Precious Efeire Igein, Gabriel Chagas Boff, Lawrence Okon, Mbuyazwe France Dube, Mike Brignol, Olanrewaju Ayomide Adebayo, Robert Yamashita, Rodrigo Alejandro Pacheco Jorquera, Tyler Matthew Peterson, Vinicius Lima Alves, Wadzanai Murombedzi.

Proposed Application Name: BookVerse API

Application Info

What will the API do?

The BookVerse API is a streamlined backend cataloging engine designed to manage an online bookstore's central inventory, authors indices, and user reading orders. It provides developers and simple frontends with clear, accessible routes to perform standard CRUD operations for browsing books, grouping items by generic literary genres, and compiling customer purchase records.

How will your API utilize a login system?

The API implements basic OAuth 2.0 integration (via Google OAuth or Auth0) coupled with stateless JSON Web Tokens (JWT). When an end-user connects securely via a login screen, the system verifies profile registration details in our database indices and builds an encrypted access token payload. This token is used to handle fundamental role boundaries, ensuring standard consumers can only purchase or browse books, while administrators maintain exclusive access to stock controls and catalog updates.

What database will you use?

We are utilizing MongoDB Atlas (a cloud-hosted NoSQL database environment) connected natively using the official `mongodb` driver or Mongoose ODM layers. MongoDB's document-centric database approach easily stores simple entity representations without multi-table join structures, letting us focus on rapid route delivery and fast query execution.

How will the data be stored in your database?

Data is partitioned across four foundational collections:

- **users:** Stores core customer data models, email addresses, and systemic authentication role configurations.

- **books:** Stores title strings, ISBN tags, descriptive text, prices, stock counts, and related author string mappings.
- **authors:** Holds author biographical contexts, names, genres, and cross-referenced book collection ID arrays.
- **orders:** Maintains client purchase line items, totals, timestamps, and order completion indicators.

How would a frontend be able to manage authentication state based on the data you provide?

Upon successful verification, the server transmits a signed JWT containing basic non-sensitive identifier definitions and expiration variables, along with profile configuration fields. The client application interceptor securely caches this string in system memory or `HttpOnly` cookies. The frontend decodes the token claims locally to identify active account states, adjust rendering flags, and attach the authorization credentials safely to subsequent network request headers via the `Authorization: Bearer <token>` standard.

1. What pieces of data in your app will need to be secured?

- **User Datasets:** Customer emails, password structures, system authentication logs, and personal shipping address arrays.
- **Administrative Invoicing & Ordering:** Financial transaction summaries, order line logs, and pricing data strings.

How will you demonstrate web security principles in the development of this app?

- **Granular Routing Middlewares:** Enforcing clear middleware layers that shield sensitive update or deletion paths, blocking regular customers from modifying book data or viewing other users' orders.
- **Strict Form Filtering:** Leveraging express-validator to intercept, sanitize, and validate inbound request structures before execution, preventing NoSQL injection parameters.
- **Standard Server Controls:** Using Helmet middleware to set secure HTTP headers, implementing Cross-Origin Resource Sharing (CORS) security limits, and deploying rate-limiters to counter server strain.

What file structure and program architecture will you use for this project? Why?

We are adopting a clean Layered Architecture / MVC approach within our Node.js design pattern to clearly compartmentalize system logic tasks:

```
src/
├── config/      # Database connection setup and environment variable maps
├── controllers/ # Processes structural route requests and manages database output
objects
├── middleware/  # JSON Web Token authenticators, schema filters, and standard
exception handlers
```

```
|— models/      # Data schema blueprints and persistence interfaces
|— routes/      # Maps targeted REST URL paths directly to designated target
controllers
|— swagger/     # JSON/YAML baseline Open API standard interactive runtime
schemes
|— app.js       # Server application configuration and bootstrapping entry engine
```

Architectural Rationale: Isolating routes from database logic ensures maximum clarity. With a large engineering group of 13 people, this modular structure allows team members to build separate catalog modules independently without touching the same files, keeping Git workflows smooth and avoiding merge conflicts.

What are potential stretch challenges that you could implement to go above and beyond?

- **Advanced Keyword Search:** Implementing native MongoDB textual search patterns to index book descriptions and titles, allowing fuzzy matching across user queries.
- **Top 10 ordered books:** Create a filter to order books by the order quantity.

API Endpoint Planning

books

- **GET /books** - Returns an array list of all available books in the bookstore catalog.
- **GET /books/{id}** - Returns comprehensive detailed properties for an individual book document.
- **POST /books** - Appends a new book entry into the collection inventory index. (Admin only).
- **PUT /books/{id}** - Modifies pricing data, description text strings, or available stock counts. (Admin only).
- **DELETE /books/{id}** - Drops a book record entirely out of the store catalog collection index. (Admin only).

authors

- **GET /authors** - Fetches a list of all indexed authors within the system.
- **GET /authors/{id}** - Returns the biography and cross-referenced book identifiers for a specific author.
- **POST /authors** - Introduces a new author biographical reference row into the system index. (Admin only).
- **PUT /authors/{id}** - Edits biographical details or update name parameters. (Admin only).
- **DELETE /authors/{id}** - Removes an author document from the database catalog tracking. (Admin only).

orders

- **GET /orders** - Returns all historical purchases made by the actively authenticated customer profile.
- **POST /orders** - Compiles items from a user's shopping basket to calculate totals and create an active purchase invoice.
- **DELETE /orders/{id}** - Cancels an open order document or handles processing returns logs. (Admin only).

Project Scheduling and Delegation

Semester Task Timeline Matrix

Academic Week	Project Target Milestones & Execution Paths
Week 04 Tasks	Formal synthesis and submission of finalized comprehensive Project Proposal framework documentation.
Week 05 Tasks	Initialize collaborative Version Control repository. Setup continuous cloud distribution hosting parameters. Author interactive Open API Swagger documentation specs exposed seamlessly live at '/api-docs' path endpoints.
Week 06 Tasks	Core code sprinting: instantiate database server pools, engineer global authentication intercept engines, and build controller endpoint route configurations.
Week 07 Tasks	Run detailed logic functional verification cycles, evaluate security defenses, perform integration checks, and submit the Final Video Project Demonstration.

How will you divide up work in your team?

To manage development velocity cleanly across our extensive peer roster, we have partitioned operations into specialized functional working engineering tracks:

DevOps & Project Core Track

Members: Alexander Bredee Holden, Eric Ayanru, Favour Precious Efeire Igein

- **Tasks:** Node.js base initialization, remote repository orchestration, managing live production pipelines, and configuring global MongoDB Atlas infrastructure clusters.

Books & Authors Catalog Track

Members: Gabriel Chagas Boff, Lawrence Okon, Mbuyazwe France Dube, Mike Brignol

- **Tasks:** Programming robust data controller actions ensuring absolute accuracy across all CRUD manipulation scopes for the Books and Authors inventory modules.

Orders & Access Validation Middleware Track

Members: Olanrewaju Ayomide Adebayo, Robert Yamashita, Rodrigo Alejandro Pacheco Jorquera

- **Tasks:** Engineering checkout invoice generation routes and implementing cryptographic JSON Web Token admin role validation filtering across secure routers.

Documentation, Quality Assurance & Presentations Track

Members: Tyler Matthew Peterson, Vinicius Lima Alves, Wadzanai Murombedzi

- **Tasks:** Authoring structural Swagger specification models, building deep automation testing routines, and compiling the technical video execution walk-through showcase.

Potential Risks and Risk Mitigation Techniques

What are the risks involved with you being able to finish this project in a timely manner?

- **Branch Collision/Git Multi-Merge Overhead:** With a high-volume engineering cohort injecting code strings simultaneously, we run severe risks of triggering major merge anomalies and breaking production environments.
- **Scope Creep & Feature Detours:** Investing excessive early development sprint cycles into complex background tasks like algorithmic fraction parsing might delay standard baseline operational deliverables.

How will you mitigate or overcome these risks?

- **Strict Branch Isolation Standards:** Enforcing an immutable, decoupled Git branching strategy. Direct commits into the baseline branch are entirely locked. Merges will require two-peer code verification passes after automated testing succeeds.
- **Phased Minimal Viable Product Focus:** Prioritizing stable base-level CRUD routes as absolute gates. Complex background tasks (like third-party API parsing setups) will be cleanly treated as optional stretch enhancements to be activated strictly after core requirements pass QA.